

Phase 0 — Foundations

Think Like a Computer · DSA with JavaScript

Purpose: Before solving any LeetCode problem, build the mental model. By the end of Phase 0, you can read any block of code and estimate its cost in time and space — automatically.

Module 1 — What is Big O Notation?

Big O answers one question: if my input doubles, how much slower does my code get? It measures the shape of growth — not milliseconds, not laptop speed.

The Core Complexity Classes

Notation	Name	Example	Doubles input → work?
$O(1)$	Constant	<code>arr[0]</code> — direct index access	Same
$O(\log n)$	Logarithmic	Binary search — halving each time	Grows by 1 step
$O(n)$	Linear	Single loop through array	Doubles
$O(n \log n)$	Linearithmic	<code>Array.sort()</code> — built-in sort	Slightly more than double
$O(n^2)$	Quadratic	Nested loop — every pair	Quadruples
$O(2^n)$	Exponential	Recursive subsets	Explodes

The Two Golden Rules

Rule 1 — Drop constants: $O(2n)$ and $O(5n)$ are both just $O(n)$. We care about shape, not multiplier.

Rule 2 — Drop smaller terms: $O(n^2 + n) = O(n^2)$. The dominant term wins at large scale.

Key insight: $O(1) < O(\log n) < O(n) < O(n \log n) < O(n^2) < O(2^n)$ — memorise this order. It is the hierarchy you will reference every single day.

Module 2 — Reading Time Complexity from Loops

The most practical Phase 0 skill: look at code and immediately know its complexity without calculating.

Loop Patterns Cheatsheet

Pattern	Complexity	Signal to look for
Single loop	$O(n)$	$i++$ or $i--$ through the full array
Nested loop (same input)	$O(n^2)$	loop inside a loop on same array
Two sequential loops	$O(n)$	loop, then loop — they add, not multiply
Loop that halves	$O(\log n)$	$i = i / 2$ or $i = i * 2$ each iteration
Two different inputs	$O(n + m)$	two arrays, cannot simplify further

Code Examples

```
// O(n²) — nested loops multiply
for (let i = 0; i < n; i++) {
  for (let j = i + 1; j < n; j++) { // j starts at i+1, still O(n²)
    console.log(arr[i], arr[j]); // n(n-1)/2 operations → O(n²)
  }
}
```

```
// O(n) — sequential loops add
for (let i = 0; i < n; i++) { ... } // O(n)
for (let j = 0; j < n; j++) { ... } // O(n)
// Total: O(n + n) = O(2n) = O(n)
```

```
// O(log n) — input halves each step
let i = n;
while (i > 1) { i = Math.floor(i / 2); }
```

Mental checklist: (1) Nested or sequential? (2) Counter +1 or halving? (3) One input or two?

Module 3 — Space Complexity

Space complexity counts only the extra memory your function allocates. The input itself does not count.

Space	Pattern	Example
$O(1)$	Fixed variables only, nothing grows	let total = 0; — one variable
$O(n)$	New array or map proportional to input	const result = [] — n elements
$O(n)$	Recursion — call stack frames	countdown(n) → n stack frames

The Time vs Space Tradeoff

Most optimisations trade one axis for the other. When you propose the faster solution in an interview, always say: 'I am trading $O(n)$ space for $O(n)$ time using a Map/Set.'

```
// O(n2) time, O(1) space
for (let i = 0; i < arr.length; i++)
  for (let j = i+1; j < arr.length; j++)
    if (arr[i] === arr[j]) return true;

// O(n) time, O(n) space ← preferred in interviews
const seen = new Set();
for (let x of arr) {
  if (seen.has(x)) return true;
  seen.add(x);
}
```

Space and time are independent axes. A function can be $O(n^2)$ time but $O(1)$ space — or $O(n)$ time and $O(n)$ space. Never assume one implies the other.

Module 4 — Recursion and the Call Stack

Every recursive call pushes a frame onto the call stack. The function cannot return until all frames below it return first.

The Two Rules of Recursion

- Rule 1 — Every recursive function needs a base case. No base case = stack overflow.
- Rule 2 — Each call must move closer to the base case. `countdown(n) → countdown(n-1)`. It shrinks every time.

The Critical Distinction

```
// Work BEFORE recursive call → executes on the way DOWN
function countdown(n) {
  if (n === 0) return;
  console.log(n);      // prints 3, 2, 1
  countdown(n - 1);
}

// Work AFTER recursive call → executes on the way BACK UP
function mystery(n) {
  if (n === 0) return;
  mystery(n - 1);
}
```

```
console.log(n);    // prints 1, 2, 3
}
```

This distinction matters for trees: pre-order (process node before children = on the way down) vs post-order (process after children = on the way back up).

Space cost of recursion

countdown(n) uses $O(n)$ space from the call stack alone — even though it creates no arrays or objects. Always count this in your space complexity answer.

Module 5 — JS Built-in Complexity

Know these by heart. Accidentally calling an $O(n)$ method inside a loop is the most common 'invisible $O(n^2)$ ' mistake in frontend interviews.

Array

Operation	Method	Cost	Why
Read by index	arr[i]	$O(1)$	Direct memory access
Push to end	arr.push(x)	$O(1)$	Appends in place
Pop from end	arr.pop()	$O(1)$	Removes last in place
Insert at start	arr.unshift(x)	$O(n)$	Shifts every element right — trap!
Remove from start	arr.shift()	$O(n)$	Shifts every element left — trap!
Search by value	arr.includes(x)	$O(n)$	Scans every element — avoid in loops
Sort	arr.sort()	$O(n \log n)$	Built-in comparison sort
Slice	arr.slice()	$O(n)$	Copies elements into new array

Map & Set

Operation	Map method	Set method	Cost
Insert	map.set(k, v)	set.add(x)	$O(1)$
Lookup / check	map.get(k)	set.has(x)	$O(1)$
Existence check	map.has(k)	set.has(x)	$O(1)$
Delete	map.delete(k)	set.delete(x)	$O(1)$
Iterate all	map.forEach()	set.forEach()	$O(n)$

The golden swap: nested loop $O(n^2)$ → single loop + Set/Map = $O(n)$. This is the single most common optimisation in frontend DSA interviews.

The Invisible $O(n^2)$ Trap

```
// TRAP — looks like O(n), actually O(n^2)
function dedupe(arr) {
  const result = [];
  for (let x of arr) {
    if (!result.includes(x)) // includes() is O(n) inside O(n) loop!
      result.push(x);
  }
}

// CORRECT — O(n) time, O(n) space
function dedupe(arr) {
  return [...new Set(arr)]; // Set.has() is O(1). Done.
}
```

Module 6 — The UMPIRE Framework

UMPIRE is how you behave in the first 5–8 minutes of any interview problem. Interviewers score communication as heavily as the solution itself.

Letter	Step	What you do
U	Understand	Restate the problem in your own words. Ask clarifying questions about edge cases (empty input, negatives, duplicates, special characters).
M	Match	Identify the pattern: 'Have I seen this before?' → Set. 'Count frequency?' → Map. 'Find a pair?' → Two pointers. 'Sorted array?' → Binary search.
P	Plan	Talk through pseudocode before touching the keyboard. State your approach and its complexity estimate.
I	Implement	Write the actual code. Only after U, M, P are done.
R	Review	Trace through a small example manually. Then check edge cases: empty input, single element, all duplicates, negatives.
E	Evaluate	State time AND space complexity clearly: 'O(n) time, O(n) space because the Map stores up to n entries.'

Pattern Matching Guide (for the M step)

Problem signals	Reach for
'Have I seen this before?' / duplicates / existence check	Set — $O(1)$ lookup
'How many times does X appear?' / frequency	Map — $O(1)$ read/write
'Find a pair that sums to X' / two elements	Two pointers or Map
'Longest/shortest subarray with condition'	Sliding window
'Sorted array, find target'	Binary search $O(\log n)$
'All combinations/subsets'	Backtracking / recursion
'Tree traversal' / DOM-like structure	DFS or BFS

Interview reality: A correct solution with poor communication scores lower than a slightly imperfect solution with excellent UMPIRE walkthrough. Practice saying your thinking out loud — even when solving alone.

Phase 0 — Master Summary

Module	Core concept	One-line rule
1 — Big O	Measures growth, not speed	Drop constants and smaller terms
2 — Loop analysis	Read patterns, not calculations	Nested = multiply. Sequential = add. Halving = $\log n$
3 — Space	Count extra memory only	Time and space are independent axes
4 — Recursion	Call stack frames cost space	Before call = way down. After call = way back up
5 — JS builtins	Know costs of Array, Map, Set	includes() inside a loop = invisible $O(n^2)$
6 — UMPIRE	Structure your interview behaviour	Understand → Match → Plan → Implement → Review → Evaluate

Phase 0 complete. You now have the mental model. Phase 1 begins: core data structures — arrays, strings, hash maps, stacks, queues. Every problem you solve from here uses what you built in Phase 0.